Individual Project Presentation

Machine Learning, 2026

Viktor Jáger

jager.v@gmail.com
Student ID 12698689
MSc Artificial Intelligence

Code available at: https://colab.research.google.com/drive/1z-kHtHrb9raUOzKKsaRvdCmS93yDk2vx?usp=sharing

1. Introduction & Objective

Project's main goals

- Demonstrate how real-world problems can be tackled with ML techniques
- Build a NN for classifying images of handwritten digits
- Evaluate the model's efficacy
- Critically discuss the solution

Context

- Computer vision
  - Image recognition
    - rule-based
    - statistical / ML
      - **Image classification**
- Image classification has evolved from rule-based systems to data-driven deep learning approaches
- CNNs now providing state-of-the-art performance by learning spatial patterns directly from image data

2. Data Specifics

MNIST dataset

- 60 000 training / 10 000 test image

Modified National Institute of Standards and Technology database

- 28 x 28 grayscale

size normalised, centered

- 10 classes (0, 1, … 9 digits)

3. Approach & Preprocess

Preprocessing

- Dense neural network (NN) for benchmark
  - flattening each image into a one-dimensional vector of 784 values
- Convolutional neural network (CNN) for comparisson
  - the original two-dimensional image structure is preserved, and a channel dimension is added.
- Converting pixel values to floating-point format and normalising them to the range from 0 to 1

Environment

- Google Colab
- Python
- chatGPT 5.3
- Gemini 2.5 Flash

4. Validation Split

- Original training set consists of 60 000 images

partitioning this using an 80/20 split

- yields a **training set of 48 000** and **validation set of 12 000** images
  - with stratification to preserve class balance

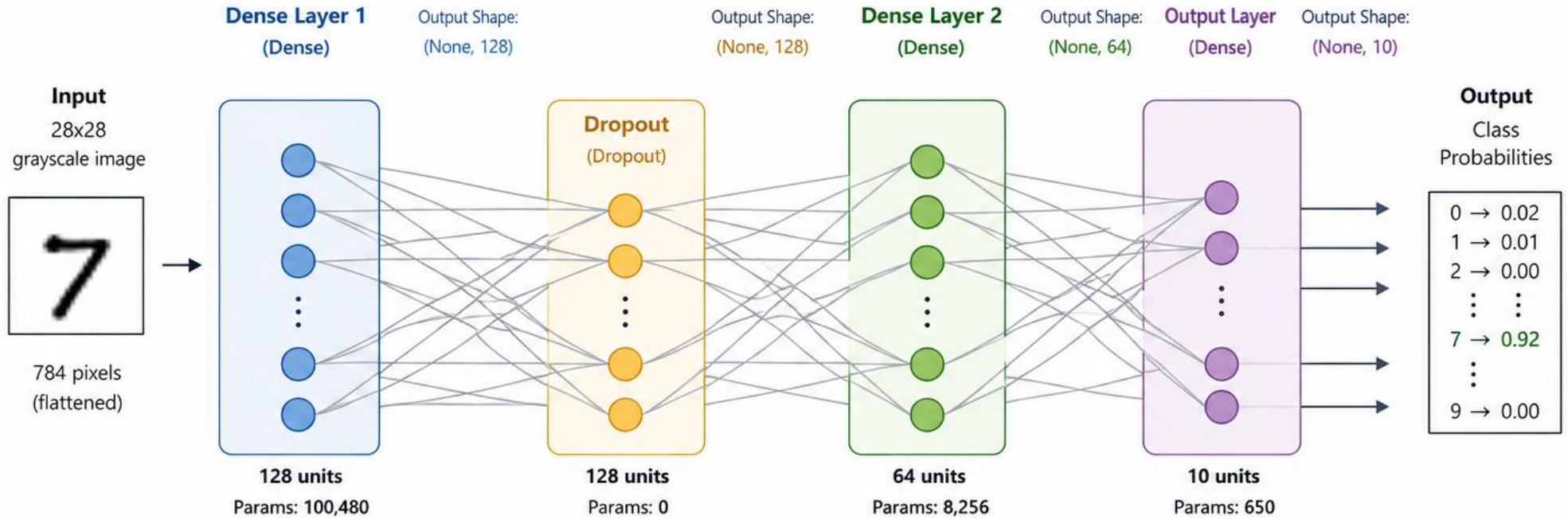
- **Validation set** is important **for monitoring** model performance during training and for supporting model comparison, while the **test set must remain untouched** until the final evaluation.
- Validation
  - Prevents overfitting
  - Enables model tuning
  - Keeps test set unbiased

5. Dense NN Architecture

- As a baseline, a simple fully connected (dense) neural network is implemented
- As for input each 28×28 image is flattened into a one-dimensional vector of 784
  - meaning that spatial relationships between pixels are not explicitly preserved
- The model consists of two hidden layers with ReLU activation functions
  - allowing to learn non-linear patterns
- Dropout (20%) was used to reduce overfitting

Dense Neural Network Architecture

(Model: sequential_1)



Layer (type)	Output Shape	Param #	Description
dense_3 (Dense)	(None, 128)	100,480	Fully connected layer with 128 neurons Input: 784 (flattened pixels) → Output: 128
dropout_1 (Dropout)	(None, 128)	0	Dropout layer (regularization) No parameters
dense_4 (Dense)	(None, 64)	8,256	Fully connected layer with 64 neurons Input: 128 → Output: 64
dense_5 (Dense)	(None, 10)	650	Output layer with 10 neurons (one per class) Softmax activation → class probabilities

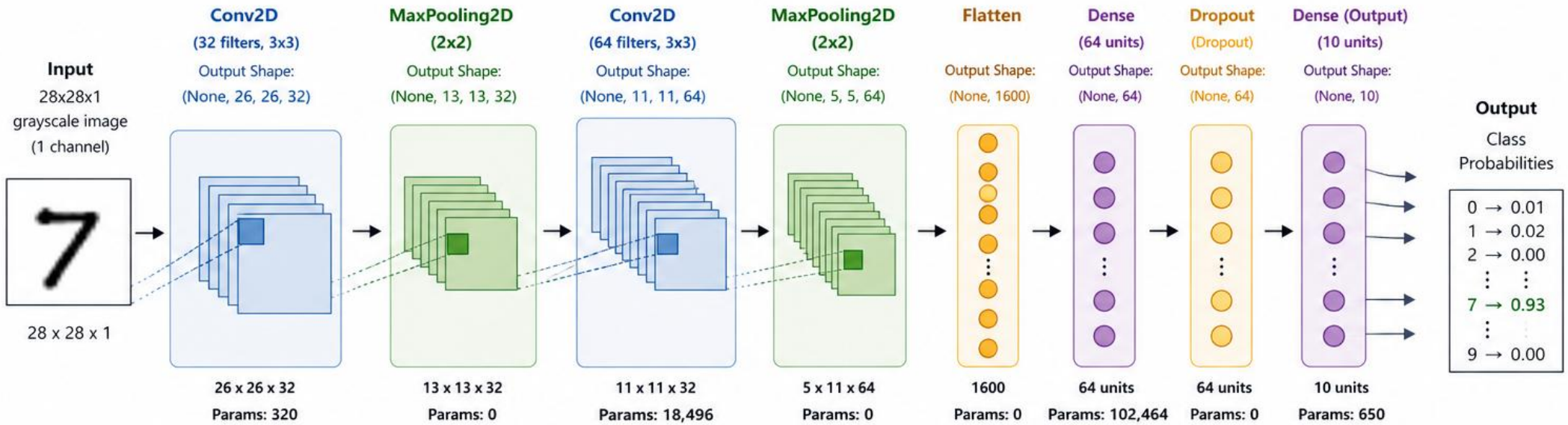
Total Trainable Parameters: 109,386

6. CNN Architecture

- This model uses convolutional layers to extract local visual features
- Followed by max-pooling layers to reduce dimensionality while retaining the most important information.
- After feature extraction, the output is flattened and passed through a dense layer before the final classification step
- Dropout (20%) was used to reduce overfitting
- The CNN preserves spatial information and learns local patterns

Convolutional Neural Network (CNN) Architecture

(Model: sequential_2)



Layer (type)	Output Shape	Param #	Description
conv2d (Conv2D)	(None, 26, 26, 32)	320	2D convolution with 32 filters of size 3x3, stride 1, valid padding Input: 28x28x1 → Output: 26x26x32
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0	Max pooling with 2x2 window, stride 2 Reduces spatial dimensions by half
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496	2D convolution with 64 filters of size 3x3, stride 1, valid padding Input: 13x13x32 → Output: 11x11x64
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0	Max pooling with 2x2 window, stride 2 Reduces spatial dimensions by half
flatten (Flatten)	(None, 1600)	0	Flattens 5x5x64 feature maps into a 1D vector of size 1600
dense_6 (Dense)	(None, 64)	102,464	Fully connected layer with 64 neurons Input: 1600 → Output: 64
dropout_2 (Dropout)	(None, 64)	0	Dropout layer for regularization
dense_7 (Dense)	(None, 10)	650	Output layer with 10 neurons (one per class) Softmax activation → class probabilities

Total params: 121,930 (476.29 KB) | Trainable params: 121,930 | Non-trainable params: 0

7. Model Designs (1)

For comparison, the same model choices were made, where possible

Activation function: **ReLU** (Rectified Linear Unit) *ReLU(x)=max(0,x)*

- Introduces non-linearity
  - allows the model to learn complex shapes

- Very fast to compute
- Keeps gradients large for positive values
  - helps deep networks train effectively
- Dead neurons can appear

ReLU introduces non-linearity while maintaining efficient gradient flow, making it a standard choice for training deep neural networks.

7. Model Designs (2)

Loss Function: **Sparse Categorical Cross-Entropy** $Loss = -log(p_{true\ class})$

- Appropriate for multi-class classification

- Measures the difference between the predicted probability distribution and the true label.
  - If the model assigns a high probability to the correct class, the loss is low.
  - If it assigns low probability, the loss increases

Cross-entropy measures the difference between predicted and true class probabilities, while 'sparse' means the labels are stored as integers rather than one-hot vectors.

7. Model Designs (3)

Output Activation: **Softmax**

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Used for multi-class classification
  - Converts outputs into probabilities
  - Values sum to 1
  - Selects most likely class

Softmax takes the raw outputs (logits) and transforms them into a probability distribution by exponentiating and normalising them.

8. Training Setup (1)

Both models were trained using the same training configuration to ensure a fair comparison.

Optimizer: **Adam** (Adaptive Moment Estimation)

$$\theta = \theta - \alpha \cdot \frac{m}{\sqrt{v} + \epsilon}$$

- Adaptive as learning rates adjust automatically
- Moment refers to statistical metrics
  - 1st moment = mean
  - 2nd moment = variance

- Keeps track of
  - average gradient (~direction)
  - squared gradient (~uncertainty)

- In practice
  - large gradients → smaller steps
  - stable gradients → larger steps
  - faster convergence
  - more stable training

Adam combines momentum and adaptive learning rates by tracking both the mean and variance of gradients, allowing efficient and stable parameter updates.

8. Training Setup (2)

Epoch = 10

- An epoch represents one full pass through the training dataset
- 10 epochs are sufficient for MNIST, as the models converge relatively quickly
- Avoids unnecessary overfitting

Batch size = 32

- Batch size defines how many samples are processed before updating the model's parameters
- A standard choice of batch size of 32 provides a good balance between computational efficiency and stable learning
  - stable gradient
  - efficient computation

48 000 / 32 = 1500 updates per epoch

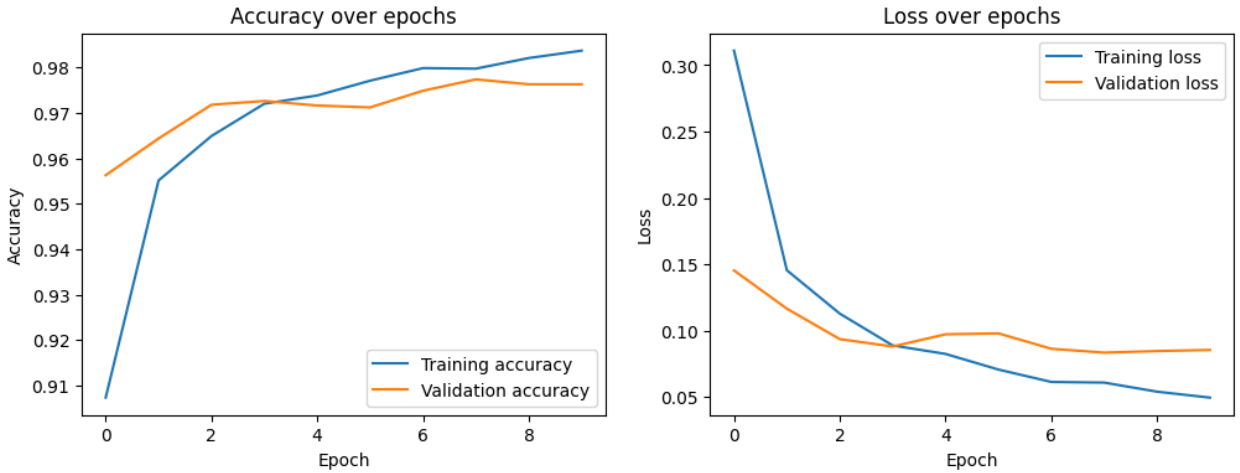
9. Validation

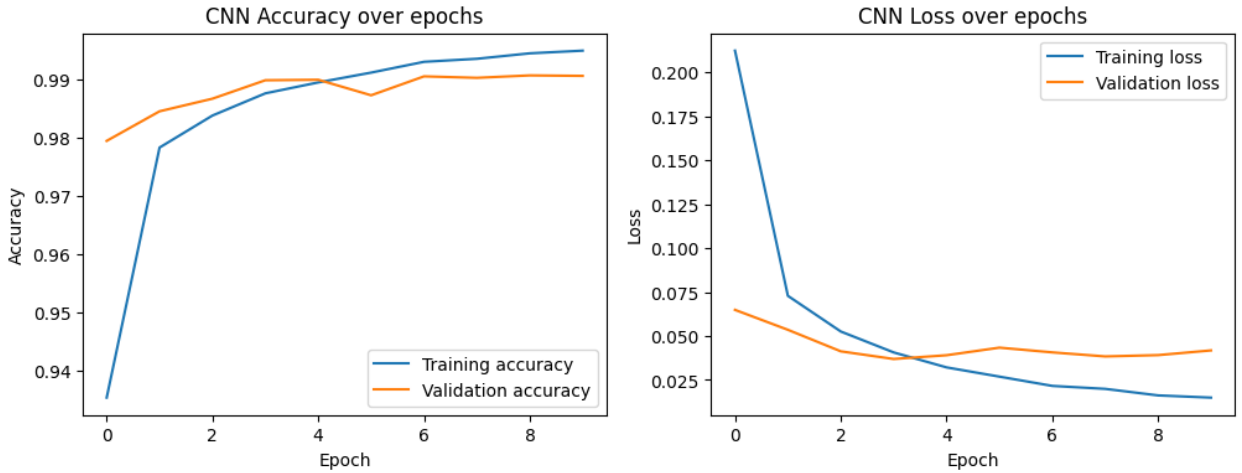
Dense NN

- Validation Accuracy = 97.6%

CNN

- Validation Accuracy = 99.0%





The CNN significantly outperformed the dense baseline.
The improvement of 1.4 percentage point is due to the CNN's ability to exploit spatial structure in the images, learning local patterns such as edges and strokes, which are essential for handwritten digit recognition.

10. Evaluation (1)

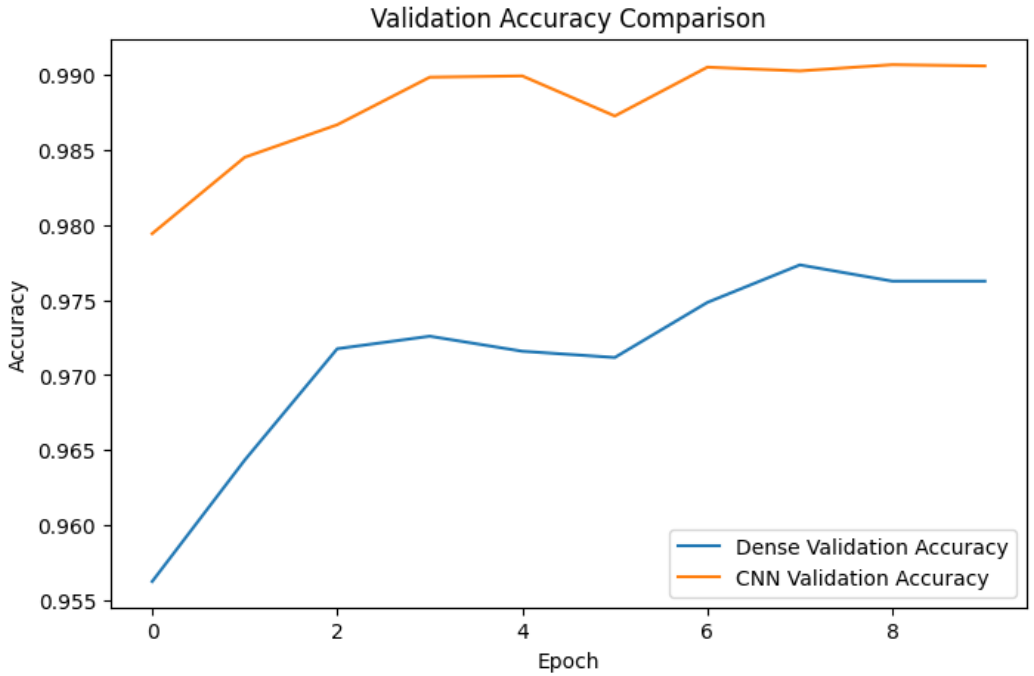
Direct comparison on the test set

Dense NN

- Test Accuracy = 0.9754
- Test Loss = 0.0807

CNN

- Test Accuracy = 907
- Test Loss = 0.0361



10. Evaluation (2)

Direct comparison on the test set

Dense NN	CNN
• Test Accuracy = 0.9754	• Test Accuracy = 0.9907
• Test Loss = 0.0807	• Test Loss = 0.0361

- The CNN outperformed the dense NN baseline and learned faster, achieving higher accuracy and better generalisation.

- This result highlights the importance of incorporating spatial structure into the model when working with image data.

- However, the performance gain comes at the cost of increased model complexity and longer training time

10. Evaluation (3)

- Training accuracies increased continuously
- Validation accuracies stabilised
- Slight increase in validation losses

} Mild overfitting

- However, the gap between training and validation performance remained limited, suggesting that generalisation is still strong

- Dropout helped mitigate overfitting in both models

11. Discussion

- The CNN model achieved the better performance, demonstrating the importance of preserving spatial structure in image data
- The dense NN model provided a useful baseline, allowing to clearly observe the benefits of the convolutional approach
- A limitation of the CNN is its increased complexity and longer training time compared to the dense model
- Although overfitting was limited, techniques such as early stopping could further improve robustness
- Future improvements could include deeper architectures or data augmentation to enhance generalisation further

Thank you for your attention

Viktor Jáger
jager.v@gmail.com